Formally Verifying **Circuits** for Zero Knowledge Proofs (ZKPs)

Alex Ozdemir

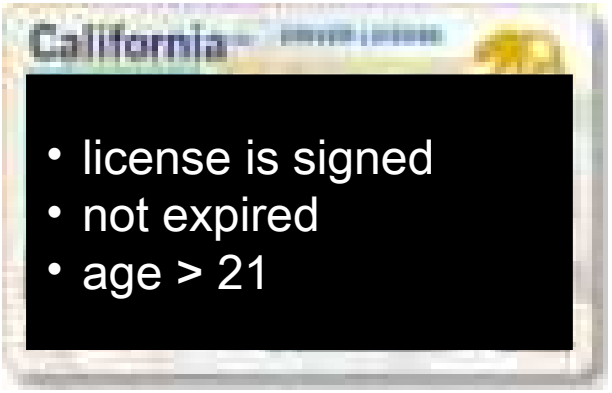
MPI-SP, Georgia Tech

Shankara Pailoor

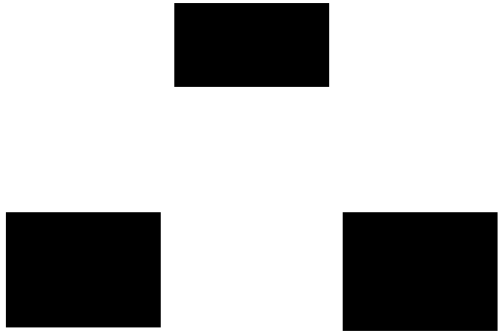
Veridise

Based on joint work with: Alp Bassa, Brian Gu, Cesare Tinelli, Clara Rodríguez, Clark Barrett, Elizaveta Pertseva, Franklyn Wang, Fraser Brown, Jacob Van-Geffen, Gereon Kremer, Hongbo Wen, Işıl Dillig, Jason Morton, Jon Stephens, Kostas Ferles, Kyle Charbonnet, Michael Chu, Riad S Wahby, Sorawee Porncharoenwase, Yanju Chen, and Yu Feng

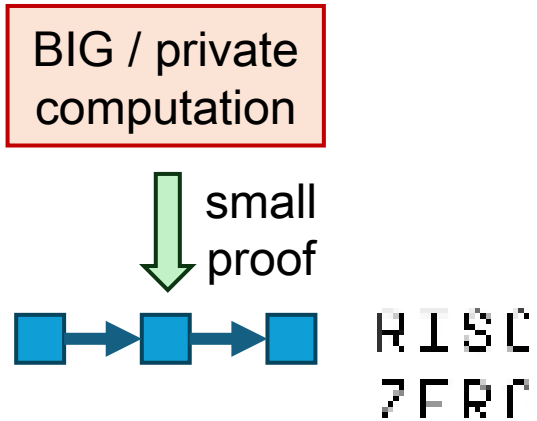
Zero Knowledge Proofs (ZKPs) are useful



Anonymous credentials



Private payments



Blockchain scalability

Aleo

Common idea: show that $P(w)$ holds

ZKPs are low-level; they need a stack

Idea

Program →

Code
(high level)

Compile →

$\mathbb{Z}_p$-circuit
(low level)

ZKP →

Application

ZKP bugs are unusually dangerous

Idea

Program →

Code
(high level)

Compile →

$\mathbb{Z}_p$-circuit
(low

tiny bug

ZKP →

Application

print money

still ZK

Three facts:

1. Vulnerabilities are **catastrophic**.
2. Exploits are dynamically **undetectable**.
3. Bugs are **real**

⇒ formal **verification** is essential

[Chaliasos et al. USENIX Security'24]

Where are the bugs?

Idea

Code (high level)

$\mathbb{Z}_p$-circuit (low level)

Application

Program → Compile → ZKP →

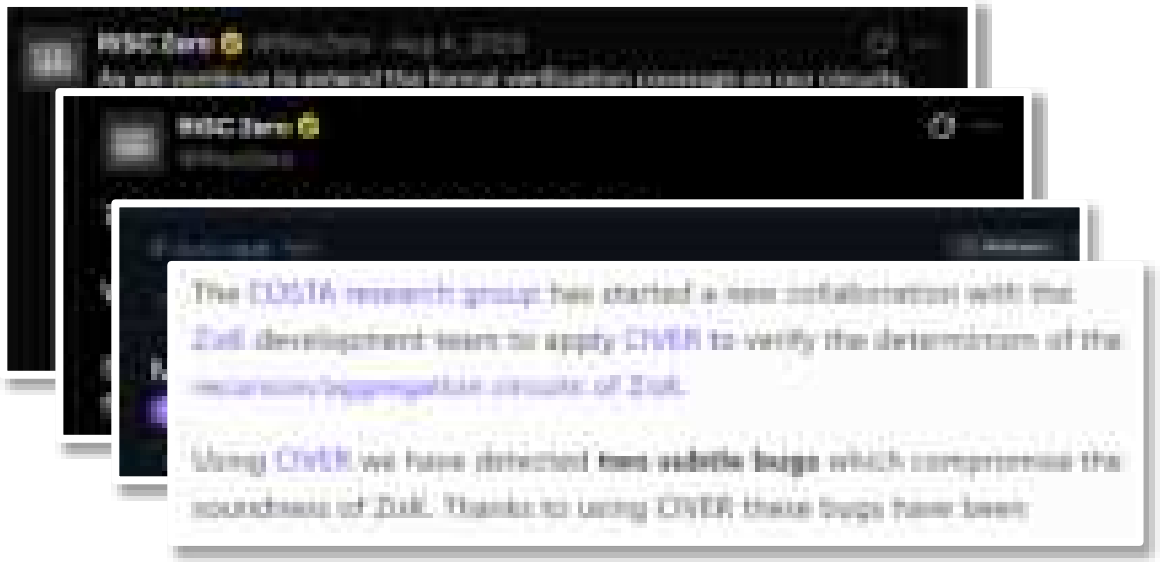
This talk: translation — 75% of bugs

Next talk: cryptography — 16% of bugs

Automated verification is already essential

> 75% of ZKP audits at Veridise use automated verification (2025)

- Finds new **bugs** that auditors miss
- Provides stronger **guarantees**
- **Integrates** into CI
- **Multiple** audit & developer teams



https://x.com/RiscZero/status/1952503598056882225 https://x.com/RiscZero/status/1910387072583774708 https://github.com/risc0/risc0/pull/3567 https://costa.fdi.ucm.es/web/news/CIVER_ZisK.html

Example: bit decomposition

Computation f (Specification)

```
fn u32_to_bits(in: u32) -> [bool; 32] {
  let mut bits = [false; 32];
  for i in 0..32 {
    bits[i] = ((in >> i) & 1) != 0;
  }
  return bits;
}
```

ZKP Circuit ϕ

$$\bigwedge_{i=0}^{31} bits_i \times (bits_i - 1) \equiv_p 0$$

$$in \equiv_p \sum_{i=0}^{31} bits_i \times 2^i$$

Are they equivalent?

Example with $in = 0$: solution: $bits_0 = 0, \ldots, bits_{31} = 0$

execution: `u32_to_bits(0) == [0, ..., 0]`

Example: bit decomposition

Computation f (Specification)

```
fn u32_to_bits(in: u32) -> [bool; 32] {
  let mut bits = [false; 32];
  for i in 0..32 {
    bits[i] = ((in >> i) & 1) != 0;
  }
  return bits;
}
```

ZKP Circuit ϕ

$$\bigwedge_{i=0}^{31} bits_i \times (bits_i - 1) \equiv_p 0$$

$$in \equiv_p \sum_{i=0}^{31} bits_i \times 2^i$$

Are they equivalent?

($p \leq 32$ bits)

It depends. What is the prime?

Example with $in = 0$: solution: $bits_0, \ldots, bits_{31} = bits\ of\ p$

execution: ☒

Non-determinism is a key challenge

Computation f (Specification)

```
fn u32_to_bits(in: u32) -> [bool; 32] {
  let mut bits = [false; 32];
  for i in 0..32 {
    bits[i] = ((in >> i) & 1) != 0;
  }
  return bits;
}
```

deterministic
computational model

`u32_to_bits(0) = [0, …, 0]`

ZKP Circuit ϕ

$$\bigwedge_{i=0}^{31} bits_i \times (bits_i - 1) \equiv_p 0$$

$$in \equiv_p \sum_{i=0}^{31} bits_i \times 2^i$$

non-deterministic
computational model

Solution 1: $in = 0, \overrightarrow{bits} = (0, \ldots, 0)$

Solution 2: $in = 0, \overrightarrow{bits} = p$'s bits

A formal verification problem

Does circuit ϕ deterministically encode computation f?

Approach: adapt existing
formal verification techniques
to ZKPs and $\mathbb{Z}_p$

relaxation

SMT solving for $\mathbb{Z}_p$

static analysis

modular reasoning

Technique 1: relaxation

Does circuit ϕ encode computation f?

In circuit ϕ, does every input $\overrightarrow{in}$ have a unique output out?

~~And does it equal $f(\overrightarrow{in})$?~~

This relaxation is called **determinism**.

It catches **96%** of bugs.

It **does not** require f.

[Chaliasos et al. USENIX Security'24]

Technique 2: SMT solving for $\mathbb{Z}_p$

- Does $\overrightarrow{in}$ uniquely determine out in circuit ϕ?
- Is this formula always true?

$$\phi \wedge \phi'$$

- Need an SMT solver for $\mathbb{Z}_p$
  - "Satisfiability Modulo ~~Theories~~ Types"
  - built with ideas from computer algebra

SMT+$\mathbb{Z}_p$ is **powerful**, but **slow**

Gröbner bases,
minimal polynomials
triangular sets,
Rabin root-finding,
split Gröbner bases,
subresultant chains,
lifting/lowering,
…

Technique 3: static analysis

Fast analyses for common patterns.

Example: bit decomposition:

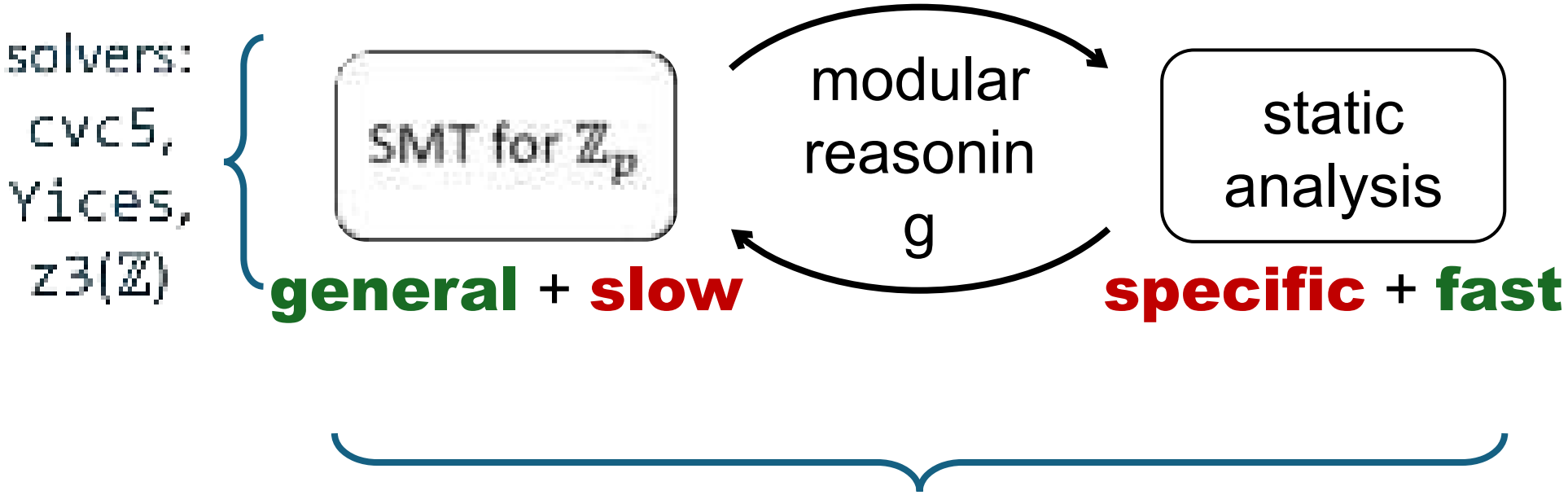
$$\bigwedge_{i=0}^{31} bits_i \times (bits_i - 1) = 0$$

$$in = \sum_{i=0}^{31} bits_i \times 2^i$$

1. Recognize pattern.
2. Check for overflow.
   - range analysis?
3. Deduce a lemma.
   - $\overrightarrow{bits} = f(in)$

Static analyses are **fast**, but **specific**

Technique 4: modular reasoning



circuit verifiers:
Picus, CIVER, Zequal, AC4, …

The road ahead

1. Other techniques:
   - Manual verification (Lean, Roqc, …)
   - Fuzzing
   - Language-independent tools
   - New properties & program logics
2. Other parts of the stack:
   - ZKP layer
   - Application layer
   - zkVMs: a related stack

Formally Verifying Circuits for ZKPs

Alex Ozdemir (MPI-SP, Georgia Tech) Shankara Pailoor (Veridise)

Idea → Program → Code → Compile → $\mathbb{F}$-circuit → ZKP → Application

bug

covert money printing

Translation phase

- lots of code & bugs (75%)
- **automatic** formal verification
  - already being used
  - **static analysis + SMT for $\mathbb{F}$**

Thank you!

Tooling Landscape

Automated Verifiers

Tool	Circuit Languages	Specification checked	Novel, real bugs?
Zequal	Circom	functional correctness (FC)	✓
Picus	Circom, Halo2, Plonky3, Gnark	determinism, limited FC	✓
CIVER	Circom	determinism, limited FC	✓
AC4	Circom, Halo2	determinism	

SMT solvers for $\mathbb{Z}_p$

Solver	Domain	Core idea	Published
cvc5		Gröbner bases + search	CAV’23
cvc5 (bitsplit)		split Gröbner bases	CAV’24
cvc5 (multimod)		cross-ring reasoning	CAV’25
YICES		model-constructing SAT	IJCAR’24
z3		simplex, S-polynomials, …	TACAS’08, CAV’24

Modular reasoning

One option: incrementality

$$in_0 \times in_1 = in_1 + x$$
$$x \times b = out$$
$$x \times (out - 1) = 0$$

Q: Is out deterministic?

Modularity lets us: **simplify**, **parallelize**, and **diversify**.

Modular reasoning

2a) incrementality

$$in_0 \times in_1 = in_1 + x$$
$$x \times b = y$$
$$x \times (y - 1) = 0$$

2b) module summaries

$$in_0 \times in_1 = in_1 + x$$
$$x \times b = y$$
$$x \times (y - 1) = 0$$

Q: Is the output y deterministic?

Modularity lets us: simplify, parallelize, and diversify.